Mô tả nghiệp vụ

Trường đại học quản lý các thiết bị dùng trong phòng dạy:

- Điều khiển máy chiếu.
- Điều khiển điều hòa.
- Micro không dây.

Mỗi thiết bị có mã, tên, trạng thái và vị trí lưu trữ. Các thiết bị khác nhau có cách kiểm tra khả năng sẵn sàng và cách tính mức ưu tiên cấp phát khác nhau.

Khi giảng viên hoặc trợ giảng yêu cầu mượn thiết bị, hệ thống cần:

1. Lưu trữ và tìm kiếm thiết bị theo mã.
2. Lọc các thiết bị phù hợp theo điều kiện.
3. Chọn một thiết bị tốt nhất theo một chiến lược lựa chọn.
4. Cấp phát thiết bị cho người mượn.
5. Trả thiết bị về kho.
6. Cho phép bổ sung loại thiết bị hoặc chiến lược chọn mới mà không sửa nhiều lớp cũ.

Tập trung vào hai quyết định thiết kế:

- Dùng **đa hình** để thay đổi hành vi theo loại thiết bị và chiến lược.
- Dùng **generic** để xây dựng kho lưu trữ và dịch vụ lựa chọn có thể làm việc với nhiều kiểu đối tượng.

1. Quy tắc nghiệp vụ

1.1. Trạng thái thiết bị

```
public enum DeviceStatus
{
    Available,
    Borrowed,
    UnderMaintenance
}
```

1.2. Quy tắc của từng loại thiết bị

Điều khiển máy chiếu

- Có thuộc tính mức pin từ 0 đến 100.
- Chỉ sẵn sàng cấp phát khi trạng thái là Available và mức pin từ 20% trở lên.
- Điểm ưu tiên mặc định bằng mức pin.

Điều khiển điều hòa

- Có thuộc tính số phòng tương thích.
- Chỉ sẵn sàng khi trạng thái là Available.
- Khi chọn cho một phòng, điều khiển phải tương thích với phòng đó.
- Điểm ưu tiên mặc định có thể bằng 100 nếu tương thích, ngược lại bằng 0.

Micro không dây

- Có thuộc tính mức pin và chất lượng tín hiệu, đều từ 0 đến 100.

- Chỉ sẵn sàng khi trạng thái là Available, pin từ 30% trở lên và tín hiệu từ 40% trở lên.
- Điểm ưu tiên mặc định bằng trung bình của pin và chất lượng tín hiệu.

2. Thiết kế trước khi lập trình

2.1. Sản phẩm thiết kế bắt buộc

Phải hoàn thành:

1. Sơ đồ lớp UML ở mức chính.
2. Xác định lớp trừu tượng và các lớp dẫn xuất.
3. Xác định interface dùng cho kho lưu trữ và chiến lược lựa chọn.
4. Chỉ ra ít nhất hai vị trí có sử dụng đa hình.
5. Chỉ ra lớp hoặc interface generic và ý nghĩa của tham số kiểu.
6. Viết ngắn gọn một quyết định thiết kế và trade-off.

Không bắt đầu cài đặt toàn bộ trước khi có bản thiết kế.

2.2. Câu hỏi định hướng thiết kế

1. Hành vi nào chung cho mọi thiết bị và nên nằm trong lớp cơ sở?
2. Hành vi nào thay đổi theo loại thiết bị và nên được ghi đè?
3. Kho lưu trữ có cần biết thiết bị cụ thể là micro hay điều khiển không?
4. Nếu có nhiều cách chọn thiết bị tốt nhất, nên dùng chuỗi if/else hay một interface chiến lược?
5. Generic constraint nào giúp bảo đảm kho chỉ lưu các đối tượng có mã định danh?
6. Có cần tạo một lớp generic cho mọi lớp trong hệ thống không? Vì sao?

3. Thiết kế lớp thiết bị và đa hình

3.1. Interface nhận diện đối tượng

```
public interface IIdentifiable
{
    string Id { get; }
}
```

Interface này cho phép các cấu trúc tổng quát tìm đối tượng theo mã mà không phụ thuộc vào lớp cụ thể.

3.2. Lớp trừu tượng TeachingDevice

```
public abstract class TeachingDevice : IIdentifiable
{
    public string Id { get; }
    public string Name { get; }
    public string StorageLocation { get; }
    public DeviceStatus Status { get; protected set; }

    public abstract bool IsReadyForUse();
    public abstract double GetDefaultPriorityScore();
```

```
    public void Borrow();
    public void Return();
}
```

Yêu cầu

- Id, Name và StorageLocation phải hợp lệ.
- Trạng thái ban đầu là Available.
- Borrow() chỉ thành công nếu IsReadyForUse() trả về true.
- Return() chỉ thành công nếu trạng thái là Borrowed.
- IsReadyForUse() và GetDefaultPriorityScore() được cài đặt đa hình ở các lớp dẫn xuất.

3.3. Các lớp dẫn xuất

```
public class ProjectorRemote : TeachingDevice
{
    public int BatteryLevel { get; }

    public override bool IsReadyForUse();
    public override double GetDefaultPriorityScore();
}
public class AirConditionerRemote : TeachingDevice
{
    public IReadOnlySet<string> CompatibleRooms { get; }

    public bool IsCompatibleWith(string roomId);
    public override bool IsReadyForUse();
    public override double GetDefaultPriorityScore();
}
public class WirelessMicrophone : TeachingDevice
{
    public int BatteryLevel { get; }
    public int SignalQuality { get; }

    public override bool IsReadyForUse();
    public override double GetDefaultPriorityScore();
}
```

4. Kho lưu trữ tổng quát

4.1. Interface generic

```
public interface IRepository<T>
    where T : IIdentifiable
{
    void Add(T item);
    bool Remove(string id);
    T? FindById(string id);
    IReadOnlyList<T> GetAll();
```

```
    IReadOnlyList<T> FindAll(Predicate<T> condition);
}
```

4.2. Lớp InMemoryRepository<T>

```
public class InMemoryRepository<T> : IRepository<T>
    where T : IIdentifiable
{
    // Cài đặt bằng List<T> hoặc Dictionary<string, T>.
}
```

4.3. Quy tắc

- Không chấp nhận phần tử null.
- Không cho phép hai phần tử trùng Id.
- Remove() trả về false nếu không tìm thấy mã.
- FindById() trả về null nếu không tìm thấy.
- GetAll() không được làm lộ collection nội bộ để mã ngoài tùy ý sửa.
- FindAll() nhận điều kiện lọc mà không phụ thuộc loại thiết bị cụ thể.

4.4. Câu hỏi thiết kế

1. Vì sao IRepository<T> cần constraint where T : IIdentifiable?
2. Nếu bỏ constraint, FindById() gặp khó khăn gì?
3. Nên sử dụng List<T> hay Dictionary<string,T>? Hãy nêu trade-off.
4. Vì sao không tạo ba lớp ProjectorRemoteRepository, AirConditionerRemoteRepository và MicrophoneRepository có mã gần giống nhau?

5. Chiến lược lựa chọn thiết bị bằng đa hình và generic

5.1. Interface chiến lược

```
public interface ISelectionStrategy<T>
{
    T Select(IReadOnlyList<T> candidates);
}
```

5.2. Các chiến lược bắt buộc

HighestPriorityStrategy<T>

Chọn đối tượng có điểm ưu tiên cao nhất.

Có thể sử dụng một hàm tính điểm được truyền từ bên ngoài:

```
public class HighestPriorityStrategy<T> : ISelectionStrategy<T>
{
    private readonly Func<T, double> scoreSelector;

    public HighestPriorityStrategy(Func<T, double> scoreSelector)
    {
        this.scoreSelector = scoreSelector;
    }
```

```
    public T Select(IReadOnlyList<T> candidates);
}
```

FirstAvailableStrategy<T>

Chọn phần tử đầu tiên trong danh sách ứng viên.

```
public class FirstAvailableStrategy<T> : ISelectionStrategy<T>
{
    public T Select(IReadOnlyList<T> candidates);
}
```

5.3. Quy tắc

- Nếu danh sách ứng viên rỗng, chiến lược phát sinh NoSuitableItemException.
- Chiến lược không thay đổi danh sách đầu vào.
- Dịch vụ cấp phát phụ thuộc vào ISelectionStrategy<T>, không phụ thuộc vào lớp chiến lược cụ thể.

6. Dịch vụ cấp phát tổng quát

```
public class AllocationService<T>
    where T : TeachingDevice
{
    private readonly IRepository<T> repository;
    private ISelectionStrategy<T> strategy;

    public AllocationService(
        IRepository<T> repository,
        ISelectionStrategy<T> strategy);

    public void ChangeStrategy(ISelectionStrategy<T> newStrategy);

    public T Allocate(Predicate<T> additionalCondition);
}
```

6.1. Yêu cầu của Allocate()

1. Lấy các đối tượng từ repository.
2. Chỉ giữ những thiết bị mà IsReadyForUse() trả về true.
3. Áp dụng additionalCondition nếu được cung cấp.
4. Dùng chiến lược hiện tại để chọn thiết bị.
5. Gọi Borrow() trên thiết bị được chọn.
6. Trả về thiết bị đã cấp phát.

6.2. Ví dụ sử dụng

Chọn điều khiển máy chiếu có pin cao nhất

```
var strategy = new HighestPriorityStrategy<ProjectorRemote>(
```

```
    device => device.GetDefaultPriorityScore());

var service = new AllocationService<ProjectorRemote>(
    projectorRepository,
    strategy);

ProjectorRemote selected = service.Allocate(device => true);
```

Chọn điều khiển điều hòa tương thích với phòng A203

```
AirConditionerRemote selected = airConditionerService.Allocate(
    device => device.IsCompatibleWith("A203"));
```

6.3. Yêu cầu thiết kế

Không viết trong AllocationService<T>:

```
if (device is ProjectorRemote) { ... }
else if (device is AirConditionerRemote) { ... }
else if (device is WirelessMicrophone) { ... }
```

Nếu bổ sung InteractiveDisplayRemote, dịch vụ cấp phát tổng quát không cần sửa mã.

7. Ngoại lệ exception

```
public class DuplicateIdException : Exception
{
    public string DuplicateId { get; }
}
public class NoSuitableItemException : Exception
{
}
public class DeviceNotReadyException : Exception
{
    public string DeviceId { get; }
    public DeviceStatus CurrentStatus { get; }
}
```

Sinh viên có thể tái sử dụng hệ phân cấp ngoại lệ của bài trước nếu phù hợp.

8. Sơ đồ lớp class diagram

Xây dựng sơ đồ

Chú ý

- TeachingDevice cung cấp cấu trúc chung, còn hành vi sẵn sàng và tính điểm được đa hình hóa.
- IRepository<T> và InMemoryRepository<T> tổng quát hóa thao tác lưu trữ.
- ISelectionStrategy<T> đa hình hóa thuật toán lựa chọn.

- AllocationService<T> phối hợp repository và strategy thông qua abstraction.
- Generic không thay thế đa hình. Hai cơ chế phối hợp với nhau:
    - Generic cung cấp **an toàn kiểu và tái sử dụng thuật toán**.
    - Đa hình cung cấp **khả năng thay đổi hành vi tại thời gian chạy**.

9. Yêu cầu kiểm thử

Tạo tối thiểu:

- 3 điều khiển máy chiếu có mức pin khác nhau.
- 2 điều khiển điều hòa với tập phòng tương thích khác nhau.
- 3 micro có pin và chất lượng tín hiệu khác nhau.

Kiểm thử:

1. Thêm thiết bị vào repository.
2. Thử thêm hai thiết bị trùng mã.
3. Tìm thiết bị theo mã.
4. Lọc các thiết bị sẵn sàng.
5. Chọn điều khiển máy chiếu có điểm cao nhất.
6. Chọn điều khiển điều hòa tương thích với một phòng.
7. Chọn micro đầu tiên sẵn sàng.
8. Đổi chiến lược trong thời gian chạy và quan sát kết quả.
9. Thử cấp phát khi không có ứng viên phù hợp.
10. Chứng minh thiết bị được chọn chuyển sang Borrowed.
11. Chứng minh repository tổng quát có thể dùng cho ít nhất hai loại thiết bị mà không sao chép mã.